

C++ Standard Library Issues to be moved in Jacksonville

Doc. no. P0888R0

Date: Revised 2018-02-12 at 00:42:19 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2164](#)⁽ⁱ⁾. What are the semantics of `vector.emplace(vector.begin(), vector.back())`?

Section: 26.3.11.5 [[vector.modifiers](#)], 26.2 [[container.requirements](#)] **Status:** [Tentatively Ready](#) **Submitter:** Howard Hinnant **Opened:** 2012-07-07 **Last modified:** 2018-01-28

Priority: 2

View all other [issues](#) in [[vector.modifiers](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

Nikolay Ivchenkov recently brought the following example on the [std-discussion](#) newsgroup, asking whether the following program well-defined:

```
#include <iostream>
#include <vector>

int main()
{
    std::vector<int> v;
    v.reserve(4);
    v = { 1, 2, 3 };
    v.emplace(v.begin(), v.back());
    for (int x : v)
        std::cout << x << std::endl;
}
```

Nikolay Ivchenkov:

I think that an implementation of vector's 'emplace' should initialize an intermediate object with `v.back()` before any shifts take place, then perform all necessary shifts and finally replace the value pointed to by `v.begin()` with the value of the intermediate object. So, I would expect the following output:

```
3
1
2
3
```

GNU C++ 4.7.1 and GNU C++ 4.8.0 produce other results:

```
2
1
2
3
```

Howard Hinnant:

I believe Nikolay is correct that vector should initialize an intermediate object with `v.back()` before any shifts take place. As Nikolay pointed out in another email, this appears to be the only way to satisfy the strong exception guarantee when an exception is not thrown by `T`'s copy constructor, move constructor, copy assignment operator, or move assignment operator as specified by 26.3.11.5 [[vector.modifiers](#)]/p1. I.e. if the `emplace` construction throws, the vector must remain unaltered.

That leads to an implementation that tolerates objects bound to the function parameter pack of the `emplace` member function may be elements or sub-objects of elements of the container.

My position is that the standard is correct as written, but needs a clarification in this area. Self-referencing `emplace` should be legal and give the result Nikolay expects. The proposed resolution of LWG [760](#) is not correct.

[2015-02 Cologne]

LWG agrees with the analysis including the assessment of LWG [760](#) and would appreciate a concrete wording proposal.

[2015-04-07 dyp comments]

The Standard currently does not require that creation of such intermediate objects is legal. 26.2.3 [[sequence.reqmts](#)] Table 100 — "Sequence container requirements" currently specifies:

Table 100 — Sequence container requirements		
Expression	Return type	Assertion/note pre-/post-condition
...		

a.emplace(p, args);	iterator	Requires: T is EmplaceConstructible into X from args. For vector and deque, T is also MoveInsertable into X and MoveAssignable. [...]
...		

The EmplaceConstructible concept is defined via allocator_traits<A>::construct in 26.2.1 [container.requirements.general] p15.5 That's surprising to me since the related concepts use the suffix Insertable if they refer to the allocator. An additional requirement such as std::is_constructible<T, Args...> is necessary to allow creation of intermediate objects.

The creation of intermediate objects also affects other functions, such as vector.insert. Since aliasing the vector is only allowed for the single-element forms of insert and emplace (see 526), the range-forms are not affected. Similarly, aliasing is not allowed for the rvalue-reference overload. See also LWG 2266.

There might be a problem with a requirement of std::is_constructible<T, Args...> related to the issues described in LWG 2461. For example, a scoped allocator adapter passes additional arguments to the constructor of the value type. This is currently not done in recent implementations of libstdc++ and libc++ when creating the intermediate objects, they simply create the intermediate object by perfectly forwarding the arguments. If such an intermediate object is then moved to its final destination in the vector, a change of the allocator instance might be required — potentially leading to an expensive copy. One can also imagine worse problems, such as run-time errors (allocators not comparing equal at run-time) or compile-time errors (if the value type cannot be created without the additional arguments). I have not looked in detail into this issue, but I'd be reluctant adding a requirement such as std::is_constructible<T, Args...> without further investigation.

It should be noted that the creation of intermediate objects currently is inconsistent in libstdc++ vs libc++. For example, libstdc++ creates an intermediate object for vector.insert, but not vector.emplace, whereas libc++ does the exact opposite in this respect.

A live demo of the inconsistent creation of intermediate objects can be found [here](#).

[2015-10, Kona Saturday afternoon]

HH: If it were easy, it'd have wording. Over the decades I have flipped 180 degrees on this. My current position is that it should work even if the element is in the same container.

TK: What's the implementation status? JW: Broken in GCC. STL: Broken in MSVS. Users complain about this every year.

MC: 526 says push_back has to work.

HH: I think you have to make a copy of the element anyway for reasons of exception safety. [Discussion of exception guarantees]

STL: vector has strong exception guarantees. Could we not just provide the Basic guarantee here.

HH: It would terrify me to relax that guarantee. It'd be an ugly, imperceptible runtime error.

HH: I agree if we had a clean slate that strong exception safety is costing us here, and we shouldn't provide it if it costs us.

STL: I have a mail here, "how can vector provide the strong guarantee when inserting in the middle".

HH: The crucial point is that you only get the strong guarantee if the exception is thrown by something other than the copy and move operations that are used to make the hole.

STL: I think we need to clean up the wording. But it does mandate currently that the self-emplacement must work, because nothings says that you can't do it. TK clarifies that a) self-emplacement must work, and b) you get the strong guarantee only if the operations for making the hole don't throw, otherwise basic. HH agrees. STL wants this to be clear in the Standard.

STL: Should it work for deque, too? HH: Yes.

HH: I will attempt wording for this.

TK: Maybe mail this to the reflector, and maybe someone has a good idea?

JW: I will definitely not come up with anything better, but I can critique wording.

Moved to Open; Howard to provide wording, with feedback from Jonathan.

[2017-01-25, Howard suggests wording]

[2018-1-26 issues processing telecon]

Status to 'Tentatively Ready' after adding a period to Howard's wording.

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify in 26.2.3 [sequence.reqmts] Table 87 — "Sequence container requirements" as indicated:

Table 87 — Sequence container requirements (in addition to container)		
Expression	Return type	Assertion/note pre-/post-condition
[...]		
a.emplace(p, args)	iterator	Requires: T is EmplaceConstructible into X from args. For vector and deque, T is also MoveInsertable into X and MoveAssignable. Effects: Inserts an object of type T constructed with std::forward<Args>(args)... before p. [Note: args may directly or indirectly refer to a value in a. — end note]

2243⁽ⁱ⁾. istream::putback problem

Section: 30.7.4.3 [[istream.unformatted](#)] **Status:** [Tentatively Ready](#) **Submitter:** Juan Soulie **Opened:** 2013-03-01 **Last modified:** 2018-01-10

Priority: 3

View all other [issues](#) in [[istream.unformatted](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

In 30.7.4.3 [[istream.unformatted](#)] / 34, when describing putback, it says that "rdbuf->sputbackc()" is called. The problem are not the obvious typos in the expression, but the fact that it may lead to different interpretations, since nowhere is specified what the required argument to sputbackc is.

It can be guessed to be "rdbuf()->sputbackc(c)", but "rdbuf()->sputbackc(char_type())" or just anything would be as conforming (or non-conforming) as the first guess.

[2017-12-12, Jonathan comments and provides wording]

Fix the bogus expression, and change sputbackc() to just sputback since we're talking about the function, not an expression sputbackc() (which isn't a valid expression any more than rdbuf->sputbackc() is). Make the corresponding change to the equivalent wording in p36 too.

[2017-12-14 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Change 30.7.4.3 [[istream.unformatted](#)] as shown:

```
basic_istream<charT, traits>& putback(char_type c);
```

-34- *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears eofbit. After constructing a sentry object, if !good() calls setstate(failbit) which may throw an exception, and return. If rdbuf() is not null, calls rdbuf()->sputbackc(c). If rdbuf() is null, or if sputbackc returns traits::eof(), calls setstate(badbit) (which may throw ios_base::failure (30.5.5.4 [[iostate.flags](#)])). [Note: This function extracts no characters, so the value returned by the next call to gcount() is 0. — end note]

-35- *Returns:* *this.

```
basic_istream<charT, traits>& unget();
```

-36- *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears eofbit. After constructing a sentry object, if !good() calls setstate(failbit) which may throw an exception, and return. If rdbuf() is not null, calls rdbuf()->sungetc(). If rdbuf() is null, or if sungetc returns traits::eof(), calls setstate(badbit) (which may throw ios_base::failure (30.5.5.4 [[iostate.flags](#)])). [Note: This function extracts no characters, so the value returned by the next call to gcount() is 0. — end note]

-37- *Returns:* *this.

2816⁽ⁱ⁾. resize_file has impossible postcondition

Section: 30.11.14.33 [[fs.op.resize_file](#)] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2016-11-07 **Last modified:** 2018-01-28

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

resize_file has this postcondition (after resolving late comment 42, see [P0489R0](#)):

```
Postcondition: file_size(p) == new_size.
```

This is impossible for an implementation to satisfy, due to the possibility of file system races. This is not actually a postcondition; rather, it is an effect that need no longer hold when the function returns.

[Issues Telecon 16-Dec-2016]

Priority 3

[2018-01-16, Jonathan provides wording]

[2018-1-26 issues processing telecon]

Status to 'Tentatively Ready'

Proposed resolution:

This wording is relative to [N4713](#).

[Drafting note: I considered a slightly more verbose form: "Causes the size in bytes of the file p resolves to, as determined by file_size (30.11.14.14 [fs.op.file_size]), to be equal to new_size, as if by POSIX truncate." but I don't think it's an improvement. The intent of the proposed wording is that if either file_size(p) or truncate(p.c_str()) would fail then an error occurs, but no call to file_size is required, and file system races might change the size before any such call does occur.]

1. Modify 30.11.14.33 [fs.op.resize_file] as indicated:

```
void resize_file(const path& p, uintmax_t new_size);
void resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept;
```

-1- ~~Postconditions: file_size(p) == new_size~~ *Effects: Causes the size that would be returned by file_size(p) to be equal to new_size, as if by POSIX truncate.*

-2- *Throws:* As specified in 30.11.6 [fs.err.report].

~~-3- Remarks: Achieves its postconditions as if by POSIX truncate().~~

2843⁽ⁱ⁾. Unclear behavior of std::pmr::memory_resource::do_allocate()

Section: 23.12.2.2 [mem.res.private] **Status:** [Ready](#) **Submitter:** Jens Maurer **Opened:** 2016-12-13 **Last modified:** 2017-11-12

Priority: 3

View all other [issues](#) in [mem.res.private].

View all issues with [Ready](#) status.

Discussion:

The specification of do_allocate() (23.12.2.2 [mem.res.private] p2+p3) says:

Returns: A derived class shall implement this function to return a pointer to allocated storage (3.7.4.2) with a size of at least bytes. The returned storage is aligned to the specified alignment, if such alignment is supported (3.11); otherwise it is aligned to max_align.

Throws: A derived class implementation shall throw an appropriate exception if it is unable to allocate memory with the requested size and alignment.

It is unclear whether a request for an unsupported alignment (e.g. larger than max_align) yields an exception or the returned storage is silently aligned to max_align.

This is [editorial issue #966](#).

[2017-01-27 Telecon]

Priority 3; Marshall to ping Pablo for intent and provide wording.

[2017-02-12 Pablo responds and provides wording]

The original intent was:

- If the alignment is supported by the implementation (see below), then it must use that alignment or else throw.
- If the alignment is not supported it must use max_align else throw.

However, the description of do_allocate might have gone stale as the aligned-allocation proposal made its way into the standard.

The understanding I take from the definition of extended alignment in (the current text of) 3.11/3 [basic.align] and "assembling an argument list" in 5.3.4/14 [expr.new] is that it is intended that, when allocating space for an object with extended alignment in a well-formed program, the alignment *will* be honored and *will not be* truncated to max_align. I think this is a change from earlier drafts of the extended-alignment proposal, where silent truncation to max_align was permitted (I could be wrong). Anyway, it seems wrong to ever ignore the alignment parameter in do_allocate().

[2017-11 Albuquerque Wednesday issue processing]

Move to Ready.

Proposed resolution:

Change the specification of do_allocate() (23.12.2.2 [mem.res.private] p2+p3) as follows:

Returns: A derived class shall implement this function to return a pointer to allocated storage (3.7.4.2) with a size of at least bytes, *aligned to the specified alignment*. ~~The returned storage is aligned to the specified alignment, if such alignment is supported; otherwise it is aligned to max_align.~~

Throws: A derived class implementation shall throw an appropriate exception if it is unable to allocate memory with the requested size and alignment.

2849⁽ⁱ⁾. Why does !is_regular_file(from) cause copy_file to report a "file already exists" error?

Section: 30.11.14.4 [fs.op.copy_file] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-12-17 **Last modified:** 2018-01-26

Priority: 2

View other [active issues](#) in [fs.op.copy_file].

View all other [issues](#) in [fs.op.copy_file].

View all issues with [Tentatively Ready](#) status.

Discussion:

30.11.14.4 [\[fs.op.copy_file\]](#)/4 says that copy_file reports "a file already exists error as specified in [fs.err.report] if" any of several error conditions exist.

It's not clear how some of those error conditions, such as !is_regular_file(from), can be sensibly described as "file already exists". Pretty much everywhere else in the filesystem specification just says "an error" without further elaboration.

[2017-01-27 Telecon]

Priority 2; Jonathan to provide updated wording.

[2018-01-16, Jonathan comments]

I said I'd provide updated wording because I wanted to preserve the requirement that the reported error is "file already exists" for some of the error cases. I no longer think that's necessary, so I think the current P/R is fine. Please note that I don't think new wording is needed.

[2018-01-23 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Edit 30.11.14.4 [\[fs.op.copy_file\]](#)/4 as indicated:

```
bool copy_file(const path& from, const path& to, copy_options options);
bool copy_file(const path& from, const path& to, copy_options options,
               error_code& ec) noexcept;
```

-4- Effects: As follows:

(4.1) — Report ~~a file already exists~~an error as specified in 30.11.6 [\[fs.err.report\]](#) if:

(4.1.1) —[...]

(4.2) —[...]

2851⁽ⁱ⁾. std::filesystem enum classes are now underspecified

Section: 30.11.9.2 [\[fs.enum.file_type\]](#), 30.11.9.3 [\[fs.enum.copy_opts\]](#), 30.11.9.6 [\[fs.enum.dir_opts\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-12-18
Last modified: 2018-01-28

Priority: 2

View all other [issues](#) in [\[fs.enum.file_type\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2678](#) stripped the numerical values of the enumerators from three enum classes in 30.11.9 [\[fs.enum\]](#); in doing so it also removed the implicit specification 1) of the bitmask elements for the two bitmask types (copy_options and directory_options) and 2) that the file_type constants are distinct.

[2017-01-27 Telecon]

Priority 2; Jonathan to work with Tim to tweak wording.

[2018-01-16, Jonathan comments]

I no longer remember what I didn't like about Tim's P/R so I think we should accept the original P/R.

[2018-1-26 issues processing telecon]

Status to 'Tentatively Ready'

Proposed resolution:

This wording is relative to [N4713](#).

1. Edit 30.11.9.2 [\[fs.enum.file_type\]](#)/1 as indicated:

This enum class specifies constants used to identify file types, with the meanings listed in Table 123. [The values of the constants are distinct.](#)

2. Edit 30.11.9.3 [\[fs.enum.copy_opts\]](#)/1 as indicated:

The enum class type copy_options is a bitmask type (20.4.2.1.4 [\[bitmask.types\]](#)) that specifies bitmask constants used to control the semantics of copy operations. The constants are specified in option groups with the meanings listed in Table 124. [The constant none represents the empty bitmask, and Constant none](#) is shown in each option group for purposes of exposition; implementations shall provide only a single definition. [Every other constant in the table represents a distinct bitmask element.](#) Calling a library function with more than a single constant for an option group results in undefined behavior.

3. Edit 30.11.9.6 [\[fs.enum.dir_opts\]](#)/1 as indicated:

The enum class type `directory_options` is a bitmask type (20.4.2.1.4 [bitmask.types]) that specifies bitmask constants used to identify directory traversal options, with the meanings listed in Table 127. The constant `none` represents the empty bitmask; every other constant in the table represents a distinct bitmask element.

2969⁽ⁱ⁾. `polymorphic_allocator::construct()` shouldn't pass `resource()`

Section: 23.12.3.2 [mem.poly.allocator.mem] **Status:** [Ready](#) **Submitter:** Pablo Halpern **Opened:** 2017-05-30 **Last modified:** 2017-11-12

Priority: 2

View other [active issues](#) in [mem.poly.allocator.mem].

View all other [issues](#) in [mem.poly.allocator.mem].

View all issues with [Ready](#) status.

Discussion:

Section 23.12.3.2 [mem.poly.allocator.mem] defines the effect of `polymorphic_allocator<T>::construct` as:

Effects: Construct a `T` object in the storage whose address is represented by `p` by uses-allocator construction with allocator `resource()` and constructor arguments `std::forward<Args>(args)...`.

The use of `resource()` is a hold-over from the LFTS, which contains a modified definition of uses-allocator construction. This revised definition was not carried over into the C++17 WP when `allocator_resource` and `polymorphic_allocator` were moved over.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Edit 23.12.3.2 [mem.poly.allocator.mem] as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-5- *Requires:* Uses-allocator construction of `T` with allocator `resource()*this` (see 23.10.8.2 [allocator.uses.construction]) and constructor arguments `std::forward<Args>(args)...` is well-formed. [*Note:* Uses-allocator construction is always well formed for types that do not use allocators. — *end note*]

-6- *Effects:* Construct a `T` object in the storage whose address is represented by `p` by uses-allocator construction with allocator `resource()*this` and constructor arguments `std::forward<Args>(args)...`.

-7- *Throws:* Nothing unless the constructor for `T` throws.

```
template <class T1, class T2, class... Args1, class... Args2>
void construct(pair<T1,T2>* p, piecewise_construct_t,
               tuple<Args1...> x, tuple<Args2...> y);
```

-8- [*Note:* This method and the `construct` methods that follow are overloads for piecewise construction of pairs (23.4.2 [pairs.pair]). — *end note*]

-9- *Effects:* Let `xprime` be a tuple constructed from `x` according to the appropriate rule from the following list. [*Note:* The following description can be summarized as constructing a `pair<T1, T2>` object in the storage whose address is represented by `p`, as if by separate uses-allocator construction with allocator `resource()*this` (23.10.8.2 [allocator.uses.construction]) of `p->first` using the elements of `x` and `p->second` using the elements of `y`. — *end note*]

[...]

[2017-06-12, Pablo comments]

The current description is correct and does not depend on changes to uses-allocator construction. It relies on the fact that `memory_resource*` is convertible to `polymorphic_allocator`.

[2017-06-13, Tim Song reopens]

While it is true that `memory_resource*` is convertible to `polymorphic_allocator`, uses-allocator construction still requires allocators, and a `memory_resource*` isn't an allocator.

To take a concrete example from the current WP, `pmr::vector<std::promise<int>>`, as specified, will be attempting to uses-allocator construct a `promise<int>` with a `memory_resource*`, but `std::promise`'s allocator-taking constructor expects something that satisfies the allocator requirements, rather than a `memory_resource*`.

[2017-06-13, Daniel and Tim restore and improve the previously proposed wording]

[2017-07 Toronto Monday issue prioritization]

Priority 2; Dietmar to check the P/R before Albuquerque.

[2017-11 Albuquerque Wednesday issue processing]

Move to Ready.

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 23.12.3.2 [\[mem.poly.allocator.mem\]](#) as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-5- *Requires*: Uses-allocator construction of T with allocator `resource()*this` (see 23.10.8.2 [\[allocator.uses.construction\]](#)) and constructor arguments `std::forward<Args>(args)...` is well-formed. [Note: Uses-allocator construction is always well formed for types that do not use allocators. — end note]

-6- *Effects*: Construct a T object in the storage whose address is represented by p by uses-allocator construction with allocator `resource()*this` and constructor arguments `std::forward<Args>(args)...`

-7- *Throws*: Nothing unless the constructor for T throws.

```
template <class T1, class T2, class... Args1, class... Args2>
void construct(pair<T1,T2>* p, piecewise_construct_t,
              tuple<Args1...> x, tuple<Args2...> y);
```

-8- [Note: This method and the construct methods that follow are overloads for piecewise construction of pairs (23.4.2 [\[pairs.pair\]](#)). — end note]

-9- *Effects*: Let $xprime$ be a tuple constructed from x according to the appropriate rule from the following list. [Note: The following description can be summarized as constructing a `pair<T1, T2>` object in the storage whose address is represented by p , as if by separate uses-allocator construction with allocator `resource()*this` (23.10.8.2 [\[allocator.uses.construction\]](#)) of p ->first using the elements of x and p ->second using the elements of y . — end note]

(9.1) — If `uses_allocator_v<T1, memory_resource*polymorphic_allocator>` is false and `is_constructible_v<T1, Args1...>` is true, then $xprime$ is x .

(9.2) — Otherwise, if `uses_allocator_v<T1, memory_resource*polymorphic_allocator>` is true and `is_constructible_v<T1, allocator_arg_t, memory_resource*polymorphic_allocator, Args1...>` is true, then $xprime$ is `tuple_cat(make_tuple(allocator_arg, resource()*this), std::move(x))`.

(9.3) — Otherwise, if `uses_allocator_v<T1, memory_resource*polymorphic_allocator>` is true and `is_constructible_v<T1, Args1..., memory_resource*polymorphic_allocator>` is true, then $xprime$ is `tuple_cat(std::move(x), make_tuple(resource()*this))`.

(9.4) — Otherwise the program is ill formed.

Let $yprime$ be a tuple constructed from y according to the appropriate rule from the following list:

(9.5) — If `uses_allocator_v<T2, memory_resource*polymorphic_allocator>` is false and `is_constructible_v<T2, Args2...>` is true, then $yprime$ is y .

(9.6) — Otherwise, if `uses_allocator_v<T2, memory_resource*polymorphic_allocator>` is true and `is_constructible_v<T2, allocator_arg_t, memory_resource*polymorphic_allocator, Args2...>` is true, then $yprime$ is `tuple_cat(make_tuple(allocator_arg, resource()*this), std::move(y))`.

(9.7) — Otherwise, if `uses_allocator_v<T2, memory_resource*polymorphic_allocator>` is true and `is_constructible_v<T2, Args2..., memory_resource*polymorphic_allocator>` is true, then $yprime$ is `tuple_cat(std::move(y), make_tuple(resource()*this))`.

(9.8) — Otherwise the program is ill formed.

2975⁽ⁱ⁾. Missing case for pair construction in scoped and polymorphic allocators

Section: 23.12.3.2 [\[mem.poly.allocator.mem\]](#), 23.13.4 [\[allocator.adaptor.members\]](#) **Status:** [Ready](#) **Submitter:** Casey Carter **Opened:** 2017-06-13 **Last modified:** 2017-11-12

Priority: 3

View other [active issues](#) in [\[mem.poly.allocator.mem\]](#).

View all other [issues](#) in [\[mem.poly.allocator.mem\]](#).

View all issues with [Ready](#) status.

Discussion:

`scoped_allocator_adaptor` ([\[allocator.adaptor.syn\]](#)) and `polymorphic_allocator` ([\[mem.poly.allocator.class\]](#)) have identical families of members named `construct`:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);

template <class T1, class T2, class... Args1, class... Args2>
void construct(pair<T1,T2>* p, piecewise_construct_t,
              tuple<Args1...> x, tuple<Args2...> y);

template <class T1, class T2>
void construct(pair<T1,T2>* p);
template <class T1, class T2, class U, class V>
void construct(pair<T1,T2>* p, U&& x, V&& y);
template <class T1, class T2, class U, class V>
void construct(pair<T1,T2>* p, const pair<U, V>& pr);
```

```
template <class T1, class T2, class U, class V>
void construct(pair<T1,T2>* p, pair<U, V>&& pr);
```

Both allocators perform `uses_allocator` construction, and therefore need special handling for pair constructions since pair doesn't specialize `uses_allocator` (tuple gets all of that magic and pair is left out in the cold). Presumably, the intent is that the `construct` overloads whose first argument is a pointer to pair capture all pair constructions. This is not the case: invoking `construct` with a pair pointer and a non-constant lvalue pair resolves to the *first* overload when it is viable: it's a better match than the pair-pointer-and-const-lvalue-pair overload. The first overload notably does not properly perform piecewise `uses_allocator` construction for pairs as intended.

[2017-07 Toronto Monday issue prioritization]

Priority 2; Marshall to work with Casey to reduce the negations in the wording.

Previous resolution [SUPERSEDED]:

1. Modify 23.12.3.2 [mem.poly allocator.mem] as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-5- *Requires*: Uses-allocator construction of `T` with `allocator resource()` (see 23.10.8.2 [allocator.uses.construction]) and constructor arguments `std::forward<Args>(args)...` is well-formed. [Note: Uses-allocator construction is always well formed for types that do not use allocators. — end note]

-6- *Effects*: Construct a `T` object in the storage whose address is represented by `p` by uses-allocator construction with `allocator resource()` and constructor arguments `std::forward<Args>(args)...`

-7- *Throws*: Nothing unless the constructor for `T` throws.

-?- Remarks: This function shall not participate in overload resolution unless `T` is not a specialization of `pair`.

2. Modify 23.13.4 [allocator.adaptor.members] as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-9- *Effects*: [...]

-?- Remarks: This function shall not participate in overload resolution unless `T` is not a specialization of `pair`.

[2017-11-02 Marshall and Casey provide updated wording]

[2017-11 Albuquerque Wednesday issue processing]

Move to Ready.

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 23.12.3.2 [mem.poly allocator.mem] as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-5- *Requires*: Uses-allocator construction of `T` with `allocator resource()` (see 23.10.8.2 [allocator.uses.construction]) and constructor arguments `std::forward<Args>(args)...` is well-formed. [Note: Uses-allocator construction is always well formed for types that do not use allocators. — end note]

-6- *Effects*: Construct a `T` object in the storage whose address is represented by `p` by uses-allocator construction with `allocator resource()` and constructor arguments `std::forward<Args>(args)...`

-7- *Throws*: Nothing unless the constructor for `T` throws.

-?- Remarks: This function shall not participate in overload resolution if `T` is a specialization of `pair`.

2. Modify 23.13.4 [allocator.adaptor.members] as indicated:

```
template <class T, class... Args>
void construct(T* p, Args&&... args);
```

-9- *Effects*: [...]

-?- Remarks: This function shall not participate in overload resolution if `T` is a specialization of `pair`.

[2989\(i\)](#). path's stream insertion operator lets you insert everything under the sun

Section: 30.11.7.6.1 [fs.path.io] **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2017-06-27 **Last modified:** 2018-01-28

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

The rules for converting a path to a narrow character sequence aren't necessarily the same as that iostreams should use. Note that this program creates a temporary path and stream inserts that, which likely destroys information.

```
#include <iostream>
#include <filesystem>
#include <string>

void foo() {
    using namespace std;
    using namespace std::experimental::filesystem::v1;
    wstring val(L"abc");
    std::cout << val;
}
```

Using [the godbolt online compiler](#) we get:

```
foo PROC
sub     rsp, 104      ; 00000068H
lea     rdx, OFFSET FLAT:$SG44895
lea     rcx, QWORD PTR val$[rsp]
call    std::basic_string<wchar_t,std::char_traits<wchar_t>,std::allocator<wchar_t>
>::basic_string<wchar_t,std::char_traits<wchar_t>,std::allocator<wchar_t> >
lea     rdx, QWORD PTR val$[rsp]
lea     rcx, QWORD PTR $T1[rs]
call    ???$?0_WU?$char_traits@_W@std@@V?$allocator@_W@1@@path@v1@filesystem@experimental@std@@QEAA@AEBV?$basic_string@_WU?$char_traits@
lea     rdx, QWORD PTR $T1[rs]
lea     rcx, OFFSET FLAT:std::cout
call    std::experimental::filesystem::v1::operator<<<char,std::char_traits<char> >
lea     rcx, QWORD PTR $T1[rs]
call    std::experimental::filesystem::v1::path::~~path
lea     rcx, QWORD PTR val$[rsp]
call    std::basic_string<wchar_t,std::char_traits<wchar_t>,std::allocator<wchar_t>
>::~basic_string<wchar_t,std::char_traits<wchar_t>,std::allocator<wchar_t> >
add     rsp, 104      ; 00000068H
ret     0
foo ENDP
```

This should either be disabled with a SFINAE constraint, use the `auto_ptr` user-defined conversion trick, or the stream insertion operators should be made "hidden friends" to prevent conversions to path from being considered here.

[2017-07 Toronto Monday issue prioritization]

Priority 2

[2017-07 Toronto Saturday afternoon]

LWG confirmed they want the hidden friend solution, Billy O'Neal to provide wording.

[2018-1-26 issues processing telecon]

Status to 'Tentatively Ready'

Proposed resolution:

This resolution is relative to [N4659](#).

1. Edit 30.11.5 [\[fs.filesystem.syn\]](#), header `<filesystem>` synopsis, as indicated:

```
// 30.11.7.6.1 [fs.path.io], path inserter and extractor
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<<(basic_ostream<charT, traits>& os, const path& p);
template <class charT, class traits>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, path& p);
```

2. Edit 30.11.7.6.1 [\[fs.path.io\]](#) as indicated:

[Drafting note: The project editor is kindly asked to consider to move sub-clause 30.11.7.6.1 [\[fs.path.io\]](#) before sub-clause 30.11.7.6 [\[fs.path.nonmember\]](#) (as a peer of it) — end drafting note]

```
template <class charT, class traits>
friend basic_ostream<charT, traits>&
operator<<<(basic_ostream<charT, traits>& os, const path& p);

[...]

template <class charT, class traits>
friend basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, path& p);
```

3. Edit 30.11.7 [\[fs.class.path\]](#) p2, class path synopsis, as indicated:

```
namespace std::filesystem {
    class path {
    public:
        [...]
        iterator begin() const;
```

```

        iterator end() const;

// 30.11.7.6.1 [fs.path.io] path inserter and extractor
template <class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const path& p);
template <class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, path& p);
};
}

```

3000⁽ⁱ⁾. `monotonic_memory_resource::do_is_equal` uses `dynamic_cast` unnecessarily

Section: 23.12.6.2 [[mem.res.monotonic.buffer.mem](#)] **Status:** [Tentatively Ready](#) **Submitter:** Pablo Halpern **Opened:** 2017-07-14 **Last modified:** 2017-11-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Section [mem.res.monotonic.buffer.mem], paragraph 11 says

```

bool do_is_equal(const memory_resource& other) const noexcept override;

Returns: this == dynamic_cast<const monotonic_buffer_resource*>(&other).

```

The `dynamic_cast` adds nothing of value. It is an incorrect cut-and-paste from an example `do_is_equal` for a more complex resource.

[2017-07-16, Tim Song comments]

The pool resource classes appear to also have this issue.

[2017-09-18, Casey Carter expands PR to cover the pool resources.]

Previous resolution: [SUPERSEDED]

1. Edit 23.12.6.2 [[mem.res.monotonic.buffer.mem](#)] as indicated:

```

bool do_is_equal(const memory_resource& other) const noexcept override;

Returns: this == dynamic_cast<const monotonic_buffer_resource*>(&other).

```

[2017-11-01 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4687](#).

1. Edit 23.12.5.4 [[mem.res.pool.mem](#)] as indicated:

```

bool synchronized_pool_resource::do_is_equal(const memory_resource& other) const noexcept override;
const memory_resource& other) const noexcept override;

Returns: this == dynamic_cast<const synchronized_pool_resource*>(&other).

```

2. Strike 23.12.5.4 [[mem.res.pool.mem](#)] paragraph 10, and the immediately preceding declaration of `unsynchronized_pool_resource::do_is_equal`.
3. Edit 23.12.6.2 [[mem.res.monotonic.buffer.mem](#)] as indicated:

```

bool do_is_equal(const memory_resource& other) const noexcept override;

Returns: this == dynamic_cast<const monotonic_buffer_resource*>(&other).

```

3002⁽ⁱ⁾. [[networking.ts](#)] `basic_socket_acceptor::is_open()` isn't noexcept

Section: 99 [[networking.ts::socket.acceptor.ops](#)] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-07-14 **Last modified:** 2017-11-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [networking.ts](#)

`basic_socket::is_open()` is noexcept, but the corresponding function on `basic_socket_acceptor` is not. This is a simple observer with a wide contract and cannot fail.

[2017-11-01 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4656](#).

1. Edit [networking.ts::socket.acceptor], class template basic_socket_acceptor synopsis, as indicated:

```
template<class AcceptableProtocol>
class basic_socket_acceptor {
public:
    [...]
    bool is_open() const noexcept;
    [...]
};
```

2. Edit 99 [networking.ts::socket.acceptor.ops] as indicated:

```
bool is_open() const noexcept;
```

-10- *Returns:* A bool indicating whether this acceptor was opened by a previous call to open or assign.

[3004^{\(i\)}](#). §[string.capacity] and §[vector.capacity] should specify time complexity for capacity()

Section: 24.3.2.4 [string.capacity], 26.3.11.3 [vector.capacity] **Status:** [Tentatively Ready](#) **Submitter:** Andy Giese **Opened:** 2017-07-24 **Last modified:** 2017-11-12

Priority: 0

View other [active issues](#) in [string.capacity].

View all other [issues](#) in [string.capacity].

View all issues with [Tentatively Ready](#) status.

Discussion:

basic_string and vector both have a capacity function that returns the size of the internally allocated buffer. This function does not specify a time complexity nor does it have an implied time complexity. However, given the complexities for data() to be $\mathcal{O}(1)$ and size() to be $\mathcal{O}(1)$, we can imagine that it's reasonable to also require capacity() to be $\mathcal{O}(1)$, since the implementation will most likely be caching the size of the allocated buffer so as to check for the need to reallocate on insertion.

[2017-11-01 Moved to Tentatively Ready after 10 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4659](#).

1. Edit 24.3.2.4 [string.capacity] as indicated:

```
size_type capacity() const noexcept;
```

-9- *Returns:* The size of the allocated storage in the string.

[-?- Complexity: Constant time.](#)

2. Edit 26.3.11.3 [vector.capacity] as indicated:

```
size_type capacity() const noexcept;
```

-1- *Returns:* The total number of elements that the vector can hold without requiring reallocation.

[-?- Complexity: Constant time.](#)

[3005^{\(i\)}](#). Destruction order of arrays by make_shared/allocate_shared only recommended?

Section: 23.11.3.6 [util.smartptr.shared.create] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2017-08-01 **Last modified:** 2017-11-12

Priority: 0

View other [active issues](#) in [util.smartptr.shared.create].

View all other [issues](#) in [util.smartptr.shared.create].

View all issues with [Tentatively Ready](#) status.

Discussion:

In [util.smartptr.shared.create]/7.9 we find this:

"When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements should be destroyed in the reverse order of their construction."

Why is this only a "should be" and not a "shall be" (or, following usual conventions for how we write requirements on the implementation, "are")? Is there some problem that means we can't require an implementation to destroy in reverse construction order in all cases?

Previous resolution: [SUPERSEDED]

This resolution is relative to [N4687](#).

1. Edit 23.11.3.6 [\[util.smartptr.shared.create\]](#) as indicated:

```
template<class T, ...>
shared_ptr<T> make_shared(args);
template<class T, class A, ...>
shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.9) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements ~~are should be~~ destroyed in ~~decreasing index order~~ **the reverse order of their construction**.

[2017-11-01, Alisdair comments and suggests wording improvement]

I dislike the change of how we specify the order of destruction, which is clear (but non-normative) from the preceding paragraph making the order of construction explicit. We are replacing that with a different specification, so now I need to match up ""decreasing index order" to "ascending order of address" to infer the behavior that is worded more directly today.

P0 to change "should be" to "are" though.

Previous resolution: [SUPERSEDED]

This resolution is relative to [N4700](#).

1. Edit 23.11.3.6 [\[util.smartptr.shared.create\]](#) as indicated:

```
template<class T, ...>
shared_ptr<T> make_shared(args);
template<class T, class A, ...>
shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.9) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements ~~are should be~~ destroyed in the reverse order of their construction.

[2017-11-01]

The rationale for the "decreasing index order" change in the original P/R suggested by Richard Smith had been presented as that user code may destroy one or more array elements and construct new ones in their place. In those cases shared_ptr has no way of knowing the construction order, so it cannot ensure that the destruction order reverses it.

Richard: Perhaps something like:

the initialized elements ~~are should be~~ destroyed in the reverse order of their **original** construction.

would work?

[2017-11-02 Moved to Tentatively Ready after 10 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4700](#).

1. Edit 23.11.3.6 [\[util.smartptr.shared.create\]](#) as indicated:

```
template<class T, ...>
shared_ptr<T> make_shared(args);
template<class T, class A, ...>
shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.9) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements ~~are should be~~ destroyed in the reverse order of their **original** construction.

[3007^{\(i\)}](#). `allocate_shared` should rebind allocator to cv-unqualified `value_type` for construction

Section: 23.11.3.6 [[util.smartptr.shared.create](#)] **Status:** [Tentatively Ready](#) **Submitter:** Glen Joseph Fernandes **Opened:** 2017-08-06 **Last modified:** 2017-11-12

Priority: 0

View other [active issues](#) in [[util.smartptr.shared.create](#)].

View all other [issues](#) in [[util.smartptr.shared.create](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

The remarks for the `allocate_shared` family of functions specify that when constructing a (sub)object of type `U`, it uses a rebound copy of the allocator `a` passed to `allocate_shared` such that its `value_type` is `U`. However `U` can be a `const` or `volatile` qualified type, and [`allocator.requirements`] specify that the `value_type` must be cv-unqualified.

[2017-11-01 Moved to Tentatively Ready after 6 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4687](#).

1. Edit 23.11.3.6 [[util.smartptr.shared.create](#)] as indicated:

```
template<class T, ...>
shared_ptr<T> make_shared(args);
template<class T, class A, ...>
shared_ptr<T> allocate_shared(const A& a, args);
```

[...]

-7- Remarks:

[...]

(7.5) — When a (sub)object of a non-array type `U` is specified to have an initial value of `v`, or `U(l...)`, where `l...` is a list of constructor arguments, `allocate_shared` shall initialize this (sub)object via the expression

(7.5.1) — `allocator_traits<A2>::construct(a2, pv, v)` or

(7.5.2) — `allocator_traits<A2>::construct(a2, pv, l...)`

respectively, where `pv` points to storage suitable to hold an object of type `U` and `a2` of type `A2` is a rebound copy of the allocator `a` passed to `allocate_shared` such that its `value_type` is [remove_cv_t<U>](#).

(7.6) — When a (sub)object of non-array type `U` is specified to have a default initial value, `make_shared` shall initialize this (sub)object via the expression `::new(pv) U()`, where `pv` has type `void*` and points to storage suitable to hold an object of type `U`.

(7.7) — When a (sub)object of non-array type `U` is specified to have a default initial value, `allocate_shared` shall initialize this (sub)object via the expression `allocator_traits<A2>::construct(a2, pv)`, where `pv` points to storage suitable to hold an object of type `U` and `a2` of type `A2` is a rebound copy of the allocator `a` passed to `allocate_shared` such that its `value_type` is [remove_cv_t<U>](#).

[...]

[3009^{\(i\)}](#). Including `<string_view>` doesn't provide `std::size/empty/data`

Section: 27.8 [[iterator.container](#)] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-08-11 **Last modified:** 2017-11-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

`basic_string_view` has `size()`, `empty()`, and `data()` members, but including `<string_view>` isn't guaranteed to give you access to the corresponding free function templates. This seems surprising.

[2017-11-01 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4687](#).

1. Edit 27.8 [[iterator.container](#)] as indicated:

-1- In addition to being available via inclusion of the `<iterator>` header, the function templates in 27.8 are available when any of the following headers are included: `<array>`, `<deque>`, `<forward_list>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<string>`, [<string_view>](#), `<unordered_map>`, `<unordered_set>`, and `<vector>`.

[3010^{\(0\)}](#). [networking.ts] uses_executor says "if a type $T::\text{executor_type}$ exists"

Section: 99 [networking.ts:async.uses.executor.trait] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-08-17 **Last modified:** 2017-11-12

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [networking.ts](#)

[async.uses.executor.trait] p1 says "if a type $T::\text{executor_type}$ exists" but we don't want it to be required to detect private or ambiguous types.

[2017-11-01 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This resolution is relative to [N4656](#).

1. Edit [networking.ts:async.uses.executor.trait] as indicated:

-1- Remark: Detects whether T has a nested `executor_type` that is convertible from `Executor`. Meets the `BinaryTypeTrait` requirements (C++Std [meta.rqmts]). The implementation provides a definition that is derived from `true_type` if `a-type the qualified-id  $T::\text{executor\_type}$  exists is valid and denotes a type` and `is_convertible<Executor,  $T::\text{executor\_type}>::\text{value}$  != false`, otherwise it is derived from `false_type`. A program may specialize this template [...].

[3013^{\(0\)}](#). (recursive_)directory_iterator construction and traversal should not be noexcept

Section: 30.11.12.1 [\[fs.dir.itr.members\]](#), 30.11.13.1 [\[fs.rec.dir.itr.members\]](#), 30.11.14.3 [\[fs.op.copy\]](#), 30.11.14.19 [\[fs.op.is_empty\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-08-23 **Last modified:** 2017-11-12

Priority: 0

View other active issues in [\[fs.dir.itr.members\]](#).

View all other issues in [\[fs.dir.itr.members\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Constructing a `(recursive_)directory_iterator` from a path requires, at a minimum, initializing its underlying `directory_entry` object with the path formed from the supplied path and the name of the first entry, which requires a potentially throwing memory allocation; every implementation I've looked at also allocates memory to store additional data as well.

Similarly, `increment()` needs to update the path stored in `directory_entry` object to refer to the name of the next entry, which may require a memory allocation. While it might conceivably be possible to postpone the update in this case until the iterator is dereferenced (the dereference operation is not `noexcept` due to its narrow contract), it seems highly unlikely that such an implementation is intended (not to mention that it would require additional synchronization as the dereference operations are `const`).

This further calls into question whether the `error_code` overloads of `copy` and `is_empty`, whose specification uses `directory_iterator`, should be `noexcept`. There might be a case for keeping the `noexcept` for `is_empty`, although that would require changes in all implementations I checked (libstdc++, libc++, and Boost). `copy` appears to be relentlessly hostile to `noexcept`, since its specification forms a path via `operator/` in two places (bullets 4.7.4 and 4.8.2) in addition to the `directory_iterator` usage. The proposed resolution below removes both.

[2017-11-03 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This wording is relative to [N4700](#).

1. Edit 30.11.12 [\[fs.class.directory_iterator\]](#), class `directory_iterator` synopsis, as indicated:

```
[...]
explicit directory_iterator(const path& p);
directory_iterator(const path& p, directory_options options);
directory_iterator(const path& p, error_code& ec) noexcept;
directory_iterator(const path& p, directory_options options,
                  error_code& ec) noexcept;
[...]

directory_iterator& operator++();
directory_iterator& increment(error_code& ec) noexcept;
```

2. Edit 30.11.12.1 [\[fs.dir.itr.members\]](#) before p2 as indicated:

```
explicit directory_iterator(const path& p);
directory_iterator(const path& p, directory_options options);
directory_iterator(const path& p, error_code& ec) noexcept;
directory_iterator(const path& p, directory_options options, error_code& ec) noexcept;
```

-2- Effects: [...]

-3- Throws: As specified in 30.11.6 [\[fs.err.report\]](#).

-4- [Note: [...] — end note]

3. Edit 30.11.12.1 [\[fs.dir.itr.members\]](#) before p10 as indicated:

```
directory_iterator& operator++();  
directory_iterator& increment(error_code& ec) noexcept;
```

-10- *Effects*: As specified for the prefix increment operation of Input iterators (24.2.3).

-11- *Returns*: *this.

-12- *Throws*: As specified in 30.11.6 [\[fs.err.report\]](#).

4. Edit 30.11.13 [\[fs.class.rec.dir.itr\]](#), class recursive_directory_iterator synopsis, as indicated:

```
[...]  
explicit recursive_directory_iterator(const path& p);  
recursive_directory_iterator(const path& p, directory_options options);  
recursive_directory_iterator(const path& p, directory_options options,  
                             error_code& ec) noexcept;  
recursive_directory_iterator(const path& p, error_code& ec) noexcept;  
[...]  
  
recursive_directory_iterator& operator++();  
recursive_directory_iterator& increment(error_code& ec) noexcept;
```

5. Edit 30.11.13.1 [\[fs.rec.dir.itr.members\]](#) before p2 as indicated:

```
explicit recursive_directory_iterator(const path& p);  
recursive_directory_iterator(const path& p, directory_options options);  
recursive_directory_iterator(const path& p, directory_options options, error_code& ec) noexcept;  
recursive_directory_iterator(const path& p, error_code& ec) noexcept;
```

-2- *Effects*: [...]

-3- *Postconditions*: [...]

-4- *Throws*: As specified in 30.11.6 [\[fs.err.report\]](#).

-5- [Note: [...] — end note]

-6- [Note: [...] — end note]

6. Edit 30.11.13.1 [\[fs.rec.dir.itr.members\]](#) before p23 as indicated:

```
recursive_directory_iterator& operator++();  
recursive_directory_iterator& increment(error_code& ec) noexcept;
```

-23- *Effects*: As specified for the prefix increment operation of Input iterators (24.2.3), except that: [...]

-24- *Returns*: *this.

-25- *Throws*: As specified 30.11.6 [\[fs.err.report\]](#).

7. Edit 30.11.5 [\[fs.filesystem.syn\]](#), header <filesystem> synopsis, as indicated:

```
namespace std::filesystem {  
  
[...]  
  
void copy(const path& from, const path& to);  
void copy(const path& from, const path& to, error_code& ec) noexcept;  
void copy(const path& from, const path& to, copy_options options);  
void copy(const path& from, const path& to, copy_options options,  
          error_code& ec) noexcept;  
  
[...]  
  
bool is_empty(const path& p);  
bool is_empty(const path& p, error_code& ec) noexcept;  
  
[...]  
}
```

8. Edit 30.11.14.3 [\[fs.op.copy\]](#) as indicated:

```
void copy(const path& from, const path& to, error_code& ec) noexcept;
```

-2- *Effects*: Equivalent to copy(from, to, copy_options::none, ec).

```
void copy(const path& from, const path& to, copy_options options);  
void copy(const path& from, const path& to, copy_options options,  
          error_code& ec) noexcept;
```

-3- *Requires*: [...]

-4- *Effects*: [...]

-5- *Throws*: [...]

-6- *Remarks:* [...]

-7- *[Example: [...] — end example]*

9. Edit 30.11.14.19 [\[fs.op.is_empty\]](#) as indicated:

```
bool is_empty(const path& p);  
bool is_empty(const path& p, error_code& ec) noexcept;
```

-1- *Effects:* [...]

-2- *Throws:* [...]

[3014^{\(i\)}](#). More noexcept issues with filesystem operations

Section: 30.11.14.4 [\[fs.op.copy_file\]](#), 30.11.14.6 [\[fs.op.create_directories\]](#), 30.11.14.31 [\[fs.op.remove_all\]](#) **Status:** [Ready](#) **Submitter:** Tim Song **Opened:** 2017-08-23
Last modified: 2017-11-12

Priority: Not Prioritized

View other [active issues](#) in [\[fs.op.copy_file\]](#).

View all other [issues](#) in [\[fs.op.copy_file\]](#).

View all issues with [Ready](#) status.

Discussion:

`create_directories` may need to create temporary paths, and `remove_all` may need to create temporary paths and/or `directory_iterators`. These operations may require a potentially throwing memory allocation.

Implementations of `copy_file` may wish to dynamically allocate the buffer used for copying when the underlying OS doesn't supply a copy API directly. This can happen indirectly, e.g., by using `<fstream>` facilities to perform the copying without supplying a custom buffer. Unless LWG wishes to prohibit using a dynamically allocated buffer in this manner, the `noexcept` should be removed.

[2017-11 Albuquerque Wednesday night issues processing]

Moved to Ready

Proposed resolution:

This wording is relative to [N4687](#).

1. Edit 30.11.5 [\[fs.filesystem.syn\]](#), header `<filesystem>` synopsis, as indicated:

```
namespace std::filesystem {  
  
    [...]  
  
    bool copy_file(const path& from, const path& to);  
    bool copy_file(const path& from, const path& to, error_code& ec) noexcept;  
    bool copy_file(const path& from, const path& to, copy_options option);  
    bool copy_file(const path& from, const path& to, copy_options option,  
                   error_code& ec) noexcept;  
  
    [...]  
  
    bool create_directories(const path& p);  
    bool create_directories(const path& p, error_code& ec) noexcept;  
  
    [...]  
  
    uintmax_t remove_all(const path& p);  
    uintmax_t remove_all(const path& p, error_code& ec) noexcept;  
  
    [...]  
}
```

2. Edit 30.11.14.4 [\[fs.op.copy_file\]](#) as indicated:

```
bool copy_file(const path& from, const path& to);  
bool copy_file(const path& from, const path& to, error_code& ec) noexcept;
```

-1- *Returns:* [...]

-2- *Throws:* [...]

```
bool copy_file(const path& from, const path& to, copy_options options);  
bool copy_file(const path& from, const path& to, copy_options options,  
               error_code& ec) noexcept;
```

-3- *Requires:* [...]

-4- *Effects:* [...]

-5- *Returns:* [...]

-6- *Throws:* [...]

-7- Complexity: [...]

3. Edit 30.11.14.6 [\[fs.op.create_directories\]](#) as indicated:

```
bool create_directories(const path& p);
bool create_directories(const path& p, error_code& ec) noexcept;
```

-1- Effects: [...]

-2- Postconditions: [...]

-3- Returns: [...]

-4- Throws: [...]

-5- Complexity: [...]

4. Edit 30.11.14.31 [\[fs.op.remove_all\]](#) as indicated:

```
uintmax_t remove_all(const path& p);
uintmax_t remove_all(const path& p, error_code& ec) noexcept;
```

-1- Effects: [...]

-2- Postconditions: [...]

-3- Returns: [...]

-4- Throws: [...]

[3015^{\(i\)}](#). `copy_options::unspecified` underspecified

Section: 30.11.14.3 [\[fs.op.copy\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-08-24 **Last modified:** 2018-01-28

Priority: 3

View other [active issues](#) in [\[fs.op.copy\]](#).

View all other [issues](#) in [\[fs.op.copy\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

30.11.14.3 [\[fs.op.copy\]](#)/4.8.2 says in the copy-a-directory case `filesystem::copy` performs:

```
for (const directory_entry& x : directory_iterator(from))
    copy(x.path(), to/x.path().filename(), options | copy_options::unspecified);
```

Presumably this does not actually mean that the implementation is free to set whatever `copy_option` element it wishes (`directories_only?` `recursive?` `create_hard_links?`), or none at all, or – since unspecified behavior corresponds to the nondeterministic aspects of the abstract machine (6.8.1 [\[intro.execution\]](#)/3) – a nondeterministically picked element for every iteration of the loop. That would be outright insane.

I'm fairly sure that what's intended here is to set an otherwise-unused bit in `options` so as to prevent recursion in the `options == copy_options::none` case.

[2017-11-08]

Priority set to 3 after five votes on the mailing list

Previous resolution: [SUPERSEDED]

This wording is relative to [N4687](#).

1. Edit 30.11.14.3 [\[fs.op.copy\]](#) p4, bullet 4.8.2 as indicated:

(4.7) — Otherwise, if `is_regular_file(f)`, then:

[...]

(4.8) — Otherwise, if

```
is_directory(f) &&
((options & copy_options::recursive) != copy_options::none ||
 options == copy_options::none)
```

then:

(4.8.1) — If `exists(t)` is false, then `create_directory(to, from)`.

(4.8.2) — Then, iterate over the files in `from`, as if by

```
for (const directory_entry& x : directory_iterator(from))
    copy(x.path(), to/x.path().filename(), options | copy_options::unspecified-in-recursive-copy);
```

where *in-recursive-copy* is an exposition-only bitmask element of *copy_options* that is not one of the elements in 30.11.9.3 [fs.enum.copy.opts].

(4.9) — Otherwise, for the signature with argument *ec*, *ec.clear()*.

(4.10) — Otherwise, no effects.

[2018-1-26 issues processing telecon]

Status to 'Tentatively Ready' after striking the words 'Exposition-only' from the added text

Proposed resolution:

This wording is relative to [N4687](#).

1. Edit 30.11.14.3 [fs.op.copy] p4, bullet 4.8.2 as indicated:

(4.7) — Otherwise, if *is_regular_file(f)*, then:

[...]

(4.8) — Otherwise, if

```
is_directory(f) &&
((options & copy_options::recursive) != copy_options::none ||
 options == copy_options::none)
```

then:

(4.8.1) — If *exists(t)* is false, then *create_directory(to, from)*.

(4.8.2) — Then, iterate over the files in *from*, as if by

```
for (const directory_entry& x : directory_iterator(from))
    copy(x.path(), to/x.path().filename(), options | copy_options::unspecified_in-recursive-copy);
```

where *in-recursive-copy* is an bitmask element of *copy_options* that is not one of the elements in 30.11.9.3 [fs.enum.copy.opts].

(4.9) — Otherwise, for the signature with argument *ec*, *ec.clear()*.

(4.10) — Otherwise, no effects.

[3017^{\(i\)}](#). list splice functions should use addressof

Section: 26.3.9.6 [forwardlist.ops], 26.3.10.5 [list.ops] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-09-05 **Last modified:** 2017-11-12

Priority: 0

View all other issues in [forwardlist.ops].

View all issues with [Tentatively Ready](#) status.

Discussion:

26.3.9.6 [forwardlist.ops] p1 and 26.3.10.5 [list.ops] p3 say *&x != this*, but should use *addressof*. We really need front matter saying that when the library says *&x* it means *std::addressof(x)*.

[2017-11-03 Moved to Tentatively Ready after 9 positive votes for P0 on c++std-lib.]

Proposed resolution:

This wording is relative to [N4687](#).

1. Edit 26.3.9.6 [forwardlist.ops] p1 as indicated:

```
void splice_after(const_iterator position, forward_list& x);
void splice_after(const_iterator position, forward_list&& x);
```

-1- Requires: position is before *begin()* or is a dereferenceable iterator in the range [*begin()*, *end()*). *get_allocator() == x.get_allocator()*. *&addressof(x)* != this.

2. Edit 26.3.10.5 [list.ops] p3 as indicated:

```
void splice(const_iterator position, list& x);
void splice(const_iterator position, list&& x);
```

-3- Requires: *&addressof(x)* != this.

3. Edit 26.3.10.5 [list.ops] p3 as indicated:

```
template <class Compare> void merge(list& x, Compare comp);
template <class Compare> void merge(list&& x, Compare comp);
```

- 22- *Requires*: comp shall define a strict weak ordering (28.7 [\[alg.sorting\]](#)), and both the list and the argument list shall be sorted according to this ordering.
- 23- *Effects*: If ([&addressof\(x\)](#) == this) does nothing; otherwise, merges the two sorted ranges [begin(), end()) and [x.begin(), x.end()). The result is a range in which the elements will be sorted in non-decreasing order according to the ordering defined by comp; that is, for every iterator i, in the range other than the first, the condition comp(*i, *(i - 1)) will be false. Pointers and references to the moved elements of x now refer to those same elements but as members of *this. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into *this, not into x.
- 24- *Remarks*: Stable (20.5.5.7 [\[algorithm.stable\]](#)). If ([&addressof\(x\)](#) != this) the range [x.begin(), x.end()) is empty after the merge. No elements are copied by this operation. The behavior is undefined if get_allocator() != x.get_allocator().
- 25- *Complexity*: At most size() + x.size() - 1 applications of comp if ([&addressof\(x\)](#) != this); otherwise, no applications of comp are performed. If an exception is thrown other than by a comparison there are no effects.

3020⁽ⁱ⁾. [networking.ts] Remove spurious nested value_type buffer sequence requirement

Section: 99 [networking.ts::buffer.reqmts] **Status:** [Tentatively Ready](#) **Submitter:** Vinnie Falco **Opened:** 2017-09-20 **Last modified:** 2018-01-22

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: [networking.ts](#)

The post-condition requirements for ConstBufferSequence and MutableBufferSequence refer to X::value_type, but no such nested type is required. The lambda expression passed to equal can use auto const& parameter types instead.

Previous resolution: [SUPERSEDED]

This wording is relative to [N4588](#).

1. Modify 99 [networking.ts::buffer.reqmts.mutablebuffersequence] Table 12 "MutableBufferSequence requirements" as indicated:

Table 12 — MutableBufferSequence requirements			
expression	return type	assertion/note	pre/post-condition
[...]			
X u(x);		post:	
		equal(net::buffer_sequence_begin(x), net::buffer_sequence_end(x), net::buffer_sequence_begin(u), net::buffer_sequence_end(u), [](const typename X::value_type auto& v1, const typename X::value_type auto& v2) { mutable_buffer b1(v1); mutable_buffer b2(v2); return b1.data() == b2.data() && b1.size() == b2.size(); })	

2. Modify 99 [networking.ts::buffer.reqmts.constbuffersequence] Table 13 "ConstBufferSequence requirements" as indicated:

Table 13 — ConstBufferSequence requirements			
expression	return type	assertion/note	pre/post-condition
[...]			
X u(x);		post:	
		equal(net::buffer_sequence_begin(x), net::buffer_sequence_end(x), net::buffer_sequence_begin(u), net::buffer_sequence_end(u), [](const typename X::value_type auto& v1, const typename X::value_type auto& v2) { const_buffer b1(v1); const_buffer b2(v2); return b1.data() == b2.data() && b1.size() == b2.size(); })	

[2017-10-19, Peter Dimov provides improved wording]

The alternative wording prevents the need for auto parameters because it takes advantage of the "convertible to" requirement.

[2017-10-20 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4588](#).

1. Modify 99 [networking.ts::buffer.reqmts.mutablebuffersequence] Table 12 "MutableBufferSequence requirements" as indicated:

Table 12 — MutableBufferSequence requirements			
expression	return type	assertion/note	pre/post-condition
[...]			
x u(x);		post: equal(net::buffer_sequence_begin(x), net::buffer_sequence_end(x), net::buffer_sequence_begin(u), net::buffer_sequence_end(u), [](const typename X::value_type& v1mutable buffer& b1, const typename X::value_type& v2mutable buffer& b2) { mutable_buffer b1(v1); mutable_buffer b2(v2); return b1.data() == b2.data() && b1.size() == b2.size(); })	

2. Modify 99 [networking.ts::buffer.reqmts.constbuffersequence] Table 13 "ConstBufferSequence requirements" as indicated:

Table 13 — ConstBufferSequence requirements			
expression	return type	assertion/note	pre/post-condition
[...]			
x u(x);		post: equal(net::buffer_sequence_begin(x), net::buffer_sequence_end(x), net::buffer_sequence_begin(u), net::buffer_sequence_end(u), [](const typename X::value_type& v1const buffer& b1, const typename X::value_type& v2const buffer& v2) { const_buffer b1(v1); const_buffer b2(v2); return b1.data() == b2.data() && b1.size() == b2.size(); })	

[3026](#)⁽¹⁾. filesystem::weakly_canonical still defined in terms of canonical(p, base)

Section: 30.11.14.39 [[fs.op.weakly_canonical](#)] Status: [Tentatively Ready](#) Submitter: Jonathan Wakely Opened: 2017-10-14 Last modified: 2018-01-22

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2956](#) fixed canonical to no longer use a base path, but weakly_canonical should have been changed too:

Effects: Using status(p) or status(p, ec), respectively, to determine existence, return a path composed by operator/= from the result of calling canonical() without a base argument and with a [...]

Since canonical doesn't accept a base argument, it doesn't make sense to talk about calling it without one.

[2017-10-16 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4687](#).

1. Change 30.11.14.39 [[fs.op.weakly_canonical](#)] as indicated:

```
path weakly_canonical(const path& p);  
path weakly_canonical(const path& p, error_code& ec);
```

-1- Returns: [...]

-2- Effects: Using status(p) or status(p, ec), respectively, to determine existence, return a path composed by operator/= from the result of calling canonical() without a base argument and with a path argument composed of the leading elements of p that exist, if any, followed by the elements of p that do not exist, if any. For the first form, canonical() is called without an error_code argument. For the second form, canonical() is called with ec as an error_code argument, and path() is returned at the first error occurrence, if any.

[...]

3030⁽⁰⁾. Who shall meet the requirements of try_lock?

Section: 33.4.5 [\[thread.lock.algorithm\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2017-11-07 **Last modified:** 2017-11-12

Priority: 0

View all other issues in [\[thread.lock.algorithm\]](#).

View all issues with [Ready](#) status.

Discussion:

33.4.5 [\[thread.lock.algorithm\]](#) says:

"If a call to try_lock() fails, unlock() shall be called for all prior arguments and there shall be no further calls to try_lock()."

We try to use "shall" for requirements on the user (e.g. as in the previous paragraph) which is absolutely not what is meant here.

[2017-11 Albuquerque Wednesday night issues processing]

Moved to Ready

Proposed resolution:

This wording is relative to [N4700](#).

1. Change 33.4.5 [\[thread.lock.algorithm\]](#) as indicated:

```
template <class L1, class L2, class... L3> int try_lock(L1&, L2&, L3&...);
```

-1- *Requires:* [...]

-2- *Effects:* Calls try_lock() for each argument in order beginning with the first until all arguments have been processed or a call to try_lock() fails, either by returning false or by throwing an exception. If a call to try_lock() fails, unlock() shall be called for all prior arguments and there shall be with no further calls to try_lock().

[...]

```
template <class L1, class L2, class... L3> void lock(L1&, L2&, L3&...);
```

-4- *Requires:* [...]

-5- *Effects:* All arguments are locked via a sequence of calls to lock(), try_lock(), or unlock() on each argument. The sequence of calls shall does not result in deadlock, but is otherwise unspecified. [Note: A deadlock avoidance algorithm such as try-and-back-off must be used, but the specific algorithm is not specified to avoid over-constraining implementations. — end note] If a call to lock() or try_lock() throws an exception, unlock() shall be called for any argument that had been locked by a call to lock() or try_lock().

3034⁽⁰⁾. P0767R1 breaks previously-standard-layout types

Section: 23.15.7.6 [\[meta.trans.other\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2017-11-12 **Last modified:** 2017-11-22

Priority: 0

View all other issues in [\[meta.trans.other\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0767R1 "Expunge POD"](#) changed the requirement for several library types from "POD" to "trivial." Since these types no longer provide/require the standard-layout portion of "POD," the change breaks:

- user classes with a member of such a type that were standard-layout in C++17
- implementations of basic_string and basic_string_view that expect character types to be both trivial and standard layout.

It appears this breakage was not intentional and not discussed in LWG.

The fix is straight-forward: apply an additional standard-layout requirement to the affected types:

- max_align_t
- the type member of specializations of aligned_storage
- the type member of specializations of aligned_union
- the char-like objects used by basic_string and basic_string_view.

(Albeit the potential for breakage with max_align_t is admittedly small.)

[2017-11-14 Moved to Tentatively Ready after 8 positive votes for P0 on c++std-lib.]

Proposed resolution:

This wording is relative to [N4700](#) + [P0767R1](#).

1. Change in 21.2.4 [\[support.types.layout\]](#) paragraph 5:

The type `max_align_t` is a trivial [standard-layout](#) type whose alignment requirement is at least as great as that of every scalar type, and whose alignment requirement is supported in every context (6.6.5 [\[basic.align\]](#)).

2. Change the table in 23.15.7.6 [\[meta.trans.other\]](#) as indicated:

`aligned_storage`

The member typedef type shall be a trivial [standard-layout](#) type suitable for use as uninitialized storage for any object whose size is at most `Len` and whose alignment is a divisor of `Align`.

`aligned_union`

The member typedef type shall be a trivial [standard-layout](#) type suitable for use as uninitialized storage for any object whose type is listed in `Types`; its size shall be at least `Len`.

3. Change 24.1 [\[strings.general\]](#) paragraph 1 as indicated:

This Clause describes components for manipulating sequences of any non-array trivial [standard-layout](#) (6.7 [\[basic.types\]](#)) type. Such types are called char-like types, and objects of char-like types are called char-like objects or simply characters.

[3035^{\(i\)}](#). `std::allocator`'s constructors should be constexpr

Section: 23.10.10 [\[default.allocator\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Geoffrey Romer **Opened:** 2017-11-11 **Last modified:** 2018-01-22

Priority: 0

View all other [issues](#) in [\[default.allocator\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

`std::allocator`'s constructors should be constexpr. It's expected to be an empty class as far as I know, so this should impose no implementation burden, and it would be useful to permit guaranteed static initialization of objects that need to hold a `std::allocator`, but don't have to actually use it until after construction.

[2017-11-25 Moved to Tentatively Ready after 7 positive votes for P0 on c++std-lib.]

Proposed resolution:

This wording is relative to [N4700](#).

1. Change in 23.10.10 [\[default.allocator\]](#) as indicated:

```
namespace std {
    template <class T> class allocator {
    public:
        using value_type = T;
        using propagate_on_container_move_assignment = true_type;
        using is_always_equal = true_type;
        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        constexpr template <class U> allocator(const allocator<U>&) noexcept;
        ~allocator();
        T* allocate(size_t n);
        void deallocate(T* p, size_t n);
    };
}
```

[3039^{\(i\)}](#). Unnecessary decay in `thread` and `packaged_task`

Section: 33.3.2.2 [\[thread.thread.constr\]](#), 33.6.10.1 [\[futures.task.members\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Stephan T. Lavavej **Opened:** 2017-11-17 **Last modified:** 2018-01-22

Priority: 0

View all other [issues](#) in [\[thread.thread.constr\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Following [P0777R1](#) "Treating Unnecessary decay", more occurrences can be fixed. When constraints are checking for the same type as `thread` or a specialization of `packaged_task`, decaying functions to function pointers and arrays to object pointers can't affect the result.

[28-Nov-2017 Moved to Tentatively Ready after five positive votes on the ML.]

Proposed resolution:

Wording relative to [N4700](#).

1. Edit 33.3.2.2 [\[thread.thread.constr\]](#) as indicated:

```
template <class F, class... Args> explicit thread(F&& f, Args&&... args);

-3- Requires: [...]
```

-4- Remarks: This constructor shall not participate in overload resolution if `decay_remove_cvref_t<F>` is the same type as `std::thread`.

2. Edit 33.6.10.1 [\[futures.task.members\]](#) as indicated:

```
template <class F>
packaged_task(F&& f);
```

-2- Requires: [...]

-3- Remarks: This constructor shall not participate in overload resolution if `decay_remove_cvref_t<F>` is the same type as `packaged_task<R(ArgTypes...)>`.

[3041^{\(i\)}](#). Unnecessary decay in `reference_wrapper`

Section: 23.14.5.1 [\[refwrap.const\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2017-12-04 **Last modified:** 2018-01-22

Priority: 0

View all other [issues](#) in [\[refwrap.const\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

Another occurrence of unnecessary decay ([P0777](#)) was introduced by the resolution of LWG [2993](#). A decayed function/array type will never be the same type as `reference_wrapper`.

[2018-01-09 Moved to Tentatively Ready after 9 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Change 23.14.5.1 [\[refwrap.const\]](#) as indicated:

```
template<class U>
reference_wrapper(U&& u) noexcept(see below);
```

-1- Remarks: Let *FUN* denote the exposition-only functions

```
void FUN(T&) noexcept;
void FUN(T&&) = delete;
```

This constructor shall not participate in overload resolution unless the expression `FUN(declval<U>())` is well-formed and `is_same_v<decay_remove_cvref_t<U>, reference_wrapper>` is false. The expression inside `noexcept` is equivalent to `noexcept(FUN(declval<U>()))`.

[3042^{\(i\)}](#). `is_literal_type_v` should be inline

Section: D.13 [\[depr.meta.types\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-12-06 **Last modified:** 2018-01-22

Priority: 0

View all other [issues](#) in [\[depr.meta.types\]](#).

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0607R0](#) forgot to look at D.13 [\[depr.meta.types\]](#) and make `is_literal_type_v` inline.

[2018-01-08 Moved to Tentatively Ready after 8 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Change D.13 [\[depr.meta.types\]](#)p1 as indicated:

-1- The header `<type_traits>` has the following addition:

```
namespace std {
    template<class T> struct is_literal_type;
    template<class T> inline constexpr bool is_literal_type_v = is_literal_type<T>::value;

    template<class> struct result_of;    // not defined
    template<class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;
    template<class T> using result_of_t = typename result_of<T>::type;

    template<class T> struct is_pod;
    template<class T> inline constexpr bool is_pod_v = is_pod<T>::value;
}
```

3043⁽⁰⁾. Bogus postcondition for filesystem_error constructor

Section: 30.11.8.1 [[fs.filesystem_error.members](#)] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2017-12-06 Last modified: 2018-01-22

Priority: 0

View all other [issues](#) in [[fs.filesystem_error.members](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

30.11.8.1 [[fs.filesystem_error.members](#)] says that constructors of filesystem_error have the postcondition that runtime_error::what() has the value what_arg.c_str(). That's obviously incorrect: these are pointers to distinct copies of the string in any sane implementation and cannot possibly compare equal.

The requirement seems suspect for a further reason: it mandates the content of the string returned by runtime_error::what(), but filesystem_error has no direct control over the construction of its indirect non-virtual base class runtime_error. Instead, what is passed to runtime_error's constructor is determined by system_error's constructor, which in many implementations is an eagerly crafted error string. This is permitted by the specification of system_error (see 22.5.7 [[syserr.syserr](#)]) but would make the requirement unimplementable.

The proposed wording below adjusts the postcondition using the formula of system_error's constructor. As an editorial change, it also replaces the postcondition tables with normal postcondition clauses, in the spirit of [editorial issue 1875](#).

[2018-01-12 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Replace 30.11.8.1 [[fs.filesystem_error.members](#)] p2-4, including Tables 119 through 121, with the following:

```
filesystem_error(const string& what_arg, error_code ec);
```

-2- *Postconditions:* code() == ec, path1().empty() == true, path2().empty() == true, and string_view(what()).find(what_arg) != string_view::npos.

```
filesystem_error(const string& what_arg, const path& p1, error_code ec);
```

-3- *Postconditions:* code() == ec, path1() returns a reference to the stored copy of p1, path2().empty() == true, and string_view(what()).find(what_arg) != string_view::npos.

```
filesystem_error(const string& what_arg, const path& p1, const path& p2, error_code ec);
```

-4- *Postconditions:* code() == ec, path1() returns a reference to the stored copy of p1, path2() returns a reference to the stored copy of p2, and string_view(what()).find(what_arg) != string_view::npos.

2. Edit 30.11.8.1 [[fs.filesystem_error.members](#)] p7 as indicated:

```
const char* what() const noexcept override;
```

-7- *Returns:* ~~A string containing runtime_error::what().~~ **An ntbs that incorporates the what_arg argument supplied to the constructor.** The exact format is unspecified. Implementations should include the system_error::what() string and the pathnames of path1 and path2 in the native format in the returned string.

3045⁽⁰⁾. atomic<floating-point> doesn't have value_type or difference_type

Section: 32.6.3 [[atomics.types.float](#)] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2017-12-11 Last modified: 2018-01-22

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The atomic<floating-point> specialization doesn't have the value_type and difference_type member typedefs, making it unusable with most of the nonmember function templates. This doesn't seem to be the intent.

[2018-01-09 Moved to Tentatively Ready after 6 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Edit 32.6.3 [[atomics.types.float](#)] after p1, class template specialization atomic<floating-point> synopsis, as indicated:

```
namespace std {
    template<> struct atomic<floating-point> {
        using value_type = floating-point;
        using difference_type = value_type;
        static constexpr bool is_always_lock_free = implementation-defined;
        [...]
    };
}
```


[3048^{\(i\)}](#). `transform_reduce(exec, first1, last1, first2, init)` discards execution policy

Section: 29.8.5 [[transform.reduce](#)] **Status:** [Tentatively Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2017-12-15 **Last modified:** 2018-01-22

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Since there exists only one common *Effects* element for both the parallel and the non-parallel form of `transform_reduce` without explicit operation parameters, the current specification of a function call `std::transform_reduce(exec, first1, last1, first2, init)` has the same effect as if the `ExecutionPolicy` would have been ignored. Presumably this effect is unintended.

[2018-01-15 Moved to Tentatively Ready after 5 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify 29.8.5 [[transform.reduce](#)] as indicated:

```
template<class InputIterator1, class InputIterator2, class T>
    T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                      InputIterator2 first2,
                      T init);
```

-?- *Effects:* Equivalent to:

```
return transform_reduce(first1, last1, first2, init, plus<>(), multiplies<>());
```

```
template<class ExecutionPolicy,
         class ForwardIterator1, class ForwardIterator2, class T>
    T transform_reduce(ExecutionPolicy&& exec,
                      ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2,
                      T init);
```

-1- *Effects:* Equivalent to:

```
return transform_reduce(std::forward<ExecutionPolicy>(exec), first1, last1, first2, init, plus<>(),
                      multiplies<>());
```

[3051^{\(i\)}](#). Floating point classifications were inadvertently changed in P0175

Section: 29.9.1 [[cmath.syn](#)] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Köppe **Opened:** 2018-01-23 **Last modified:** 2018-01-31

Priority: 0

View other [active issues](#) in [[cmath.syn](#)].

View all other [issues](#) in [[cmath.syn](#)].

View all issues with [Tentatively Ready](#) status.

Discussion:

The paper [P0175](#) was meant to be a purely editorial change to spell out the synopses of the "C library" headers. However it contained the following inadvertent normative change: The floating point classification functions `isinf`, `isfinite`, `signbit`, etc. (but not `fpclassify`) used to be specified to return "bool" in C++14. In C, those are macros, but in C++ they have always been functions. During the preparation of P0175, I recreated the function signatures copying the return type "int" from C, but failed to notice that we had already specified those functions differently, so the return type was changed to "int".

To restore the intended specification, we should change the return types of all the `is...` and `signbit` classification functions back to `bool`. Alternatively, we could decide that the return type should actually be `int`, but that would be a larger discussion.

Proposed resolution for restoring the original wording: Replace return type "int" with return type "bool" and for all the classification/comparison functions after "*classification/comparison functions*" in [[cmath.syn](#)] *except* the `fpclassify` functions.

Related previous issue was LWG [1327](#) and the corresponding [NB comment US-136](#) resolution.

[2018-01-29 Moved to Tentatively Ready after 8 positive votes on c++std-lib.]

Proposed resolution:

This wording is relative to [N4713](#).

1. Modify 29.9.1 [[cmath.syn](#)] as indicated:

```
[...]
namespace std {
    [...]
    // 29.9.4 [c.math.fpclass], classification / comparison functions
    int fpclassify(float x);
    int fpclassify(double x);
    int fpclassify(long double x);
    bool int isfinite(float x);
```

```
bool int isfinite(double x);
bool int isfinite(long double x);
bool int isinf(float x);
bool int isinf(double x);
bool int isinf(long double x);
bool int isnan(float x);
bool int isnan(double x);
bool int isnan(long double x);
bool int isnormal(float x);
bool int isnormal(double x);
bool int isnormal(long double x);
bool int signbit(float x);
bool int signbit(double x);
bool int signbit(long double x);
bool int isgreater(float x, float y);
bool int isgreater(double x, double y);
bool int isgreater(long double x, long double y);
bool int isgreaterequal(float x, float y);
bool int isgreaterequal(double x, double y);
bool int isgreaterequal(long double x, long double y);
bool int isless(float x, float y);
bool int isless(double x, double y);
bool int isless(long double x, long double y);
bool int islessequal(float x, float y);
bool int islessequal(double x, double y);
bool int islessequal(long double x, long double y);
bool int islessgreater(float x, float y);
bool int islessgreater(double x, double y);
bool int islessgreater(long double x, long double y);
bool int isunordered(float x, float y);
bool int isunordered(double x, double y);
bool int isunordered(long double x, long double y);
[...]
```

```
}
```